

boilerplate

Guide d'utilisation



backtik

Copyright © 2026 backtik Sàrl

Le code source du boilerplate est distribué sous licence MIT.

Ce document a été réalisé par Florian Salvi pour backtik Sàrl dans le cadre de son Travail de Bachelor, effectué au terme de sa formation en Ingénierie des Médias à la Haute École d'Ingénierie et de Gestion du Canton de Vaud (HEIG-VD)

Table des matières

Boilerplate	1
Installation	2
Dépendances	3
Génération du README.md	4
Génération du CHANGELOG.md	6
Suivi des dépendances	9
Génération des tests unitaires Vitest	11
Génération des tests E2E Playwright	13
Utilisation d'outils d'intelligence artificielle	15

Boilerplate

Ce boilerplate a été développé dans le cadre d'un Travail de Bachelor d'un étudiant en Ingénierie des Médias et réalisé au sein de l'agence.

Son objectif est de fournir une base de projet qui intègre divers outils et workflows destinés à faciliter et automatiser certaines tâches secondaires du développement web et/ou mobile.

Les principales solutions qui y sont intégrées sont :

- Génération automatique de la documentation technique (README.md)
- Génération automatique de CHANGELOG.md orientés clients
- Suivi automatique des dépendances
- Génération automatique des tests unitaires Vitest
- Génération semi-automatisée des tests end-to-end Playwright

Ce boilerplate est expérimental. Sa stabilité, ses performances et son adaptabilité à tout projet ne sont pas garanties.

Sa dépendance à certains outils d'intelligence artificielle impacte fortement ses performances.

Ce boilerplate ne contient que les outils d'automatisation des tâches mentionnées. Les technologies du projet elles-mêmes doivent être installées manuellement.

Installation

URL du dépôt

https://github.com/floriansalvi/TB_Boilerplate.git

Cloner le dépôt

```
git clone https://github.com/floriansalvi/TB_Boilerplate.git
```

Ou 'Use this template' > Create a new repository

Paramètres du dépôt

- Secrets and Variables > Actions > Secrets BP_OPENAI_API_KEY
- Secrets and Variables > Actions > Variable: BP_OPENAI_MODEL
- Actions > General > Workflow permissions > Read and write

Variables d'environnement

```
cp .env.example .env
```

Installation des dépendances

```
npm install
```

Installation de Mend Renovate

Pour l'automatisation du suivi des versions de dépendances, La Renovate GitHub App doit être installée :

<https://github.com/apps/renovate>

Une fois installée, il est possible de définir les dépôts concernés.

Dépendances

Ce boilerplate comporte différentes dépendances pour les diverses solutions qu'il présente.

Elles n'ont pas été installées lors de l'exécution de la commande 'npm install', voici comment les installer séparément.

OpenAI

```
npm install openai
```

Vitest

```
npm install vitest
```

Vue Test Utils

```
npm install @vue/test-utils
```

jsdom

```
npm install jsdom
```

Playwright

```
npm install @playwright/test
```

Navigateur Chromium pour Playwright

```
npx playwright install chromium
```

Génération du README.md

Introduction

Cette fonctionnalité permet de générer automatiquement un fichier README.md à partir de la structure du projet et de ses fichiers de configuration.

Lors de son exécution, le workflow analyse les différents fichiers de configuration (package.json ; composer.json ; .env.example ; Docker ; etc.) et crée un fichier 'projet-context.json' contenant toutes les informations nécessaires.

Ce contexte est ensuite transmis à un modèle de langage (LLM) qui génère le README.md final du projet.

Déclenchement

Ce workflow est déclenché lors des **Pull Requests** sur la branche 'main'. Il est possible de le déclencher en tout temps sur GitHub > Actions > Generate README.

Fichiers utilisés

- .github > workflows > generate-readme.yml
- scripts > readme-generator > analyze-project.mjs
- scripts > readme-generator > generate-readme.mjs
- scripts > readme-generator > README-template.md
- scripts > readme-generator > prompt.md

Génération du README.md

Déroulement



1. Vérifications des fichiers de configuration

Le workflow `'generate-readme.yml'` compare les fichiers de configuration entre la branche `'main'` et la Pull Request. Si aucun fichier n'a été modifié, la génération du README est ignorée.

2. Analyse du projet

Le script `'analyze-project.mjs'` analyse les différents fichiers de configuration et génère un fichier `'project-context.json'` contenant les informations nécessaires.

3. Génération

Le script `'generate-readme.mjs'` fait une requête au LLM en incluant :

- `'project-context.json'`
- le prompt
- le README.md existant si présent
- le template du README

Le LLM génère ensuite un nouveau fichier README.md.

4. Commit automatique

Si le contenu du README.md a été modifié, le workflow crée automatiquement un commit sur la branche de la Pull Request.

Génération du CHANGELOG.md

Introduction

Cette fonctionnalité permet de générer automatiquement un fichier CHANGELOG.md à partir de l'historique de commits entre deux tags.

Lors de son exécution, le workflow extrait l'historique de commits entre deux tags et crée un fichier 'commits-list.json' contenant tous les commits (id ; message ; auteur ; date).

Cette liste est ensuite transmise à un modèle de langage (LLM) qui génère la section de la nouvelle Release. Cette dernière est insérée dans le CHANGELOG.md et dans les Release Notes.

Déclenchement

Ce workflow est déclenchée lors d'une **Release GitHub**. Il est possible de le déclencher en tout temps sur Github > Actions > Generate CHANGELOG.

Fichiers utilisés

- .github > workflows > generate-changelog.yml
- scripts > changelog-generator > generate-commits-list.mjs
- scripts > changelog-generator > generate-changelog.mjs
- scripts > changelog-generator > CHANGELOG-template.md
- scripts > changelog-generator > RELEASE-template.md
- scripts > changelog-generator > prompt.md

Génération du CHANGELOG.md

Cette génération automatique dépend de plusieurs conventions.

Conventional Commits

Pour que le LLM puisse correctement interpréter les modifications apportées au projet, chaque message de commit doit suivre la structure :

`<type>[<scope>]: <description>`

Types	Description
feat	Ajout d'une fonctionnalité.
fix	Correction d'un bug, d'une erreur, d'un comportement non souhaité.
perf	Amélioration des performances (nombre de requêtes, cache, ...).
refactor	Refactorisation du code, sans modification fonctionnelle.
style	Modification de la forme du code, sans modification fonctionnelle.
test	Ajout ou modification de fichiers de tests ou de leur configuration.
docs	Ajout ou modification de la documentation.
build	Modification du système de build, de compilation, de packaging, etc ...
ci	Ajout ou modification de scripts/workflow CI/CD.
chore	Tâche interne ne correspondant à aucune autre catégorie.
revert	Annulation complète ou partielle d'une modification antérieure.

Génération du CHANGELOG.md

Keep a Changelog

La structure du CHANGELOG et des Releases suit la convention de Keep a Changelog.

Semantic Version

Le nommage des Releases / Version suit la convention SemVer qui structure les versions comme cela : <MAJOR>.<MINOR>.<PATCH>

Déroulement



1. Extraction de l'historique des commits

Le workflow 'generate-changelog.yml' exécute le script 'generate-commits-list.mjs' qui récupère tous les commits réalisés depuis le dernier tag.

2. Génération

Le script 'generate-changelog.mjs' fait une requête au LLM en incluant :

- 'commits-list.json'
- le prompt
- le template de Release

Le LLM génère ensuite le contenu de la nouvelle Release.

4. Commit automatique

Si le contenu du CHANGELOG.md a été modifié, le workflow crée automatiquement un commit sur la branche de la Release. Le contenu généré est ajouté aux Release Notes.

Suivi des dépendances

Introduction

Cette fonctionnalité permet de suivre automatiquement l'évolution des dépendances et leurs nouvelles versions, et d'obtenir une validation lorsqu'une d'entre-elles peut être mise à jour.

Mend Renovate

Une fois installé (voir Installation p.8), Mend Renovate ouvre automatiquement une issue et une Pull Request. Une fois cette dernière fusionnée, Renovate commence automatiquement à analyser le projet.

Déclenchement

Le workflow est déclenché lors de l'ouverture de **Pull Requests** ayant un titre commençant par 'renovate/'.

Fichiers utilisés

- .github > workflows > validate-renovate.yml
- renovate.json

Configuration

La configuration de Renovate peut être adaptée dans le fichier 'renovate.json'.

Dans ce dernier, plusieurs paramètres sont adaptables – périodicité de l'analyse des dépendances , âge minimal des nouvelles versions ; etc.

Suivi des dépendances

Déroulement



1. Ouverture d'une Pull Request

Lorsque Renovate détecte une dépendance dont une nouvelle version est disponible, il crée une Pull Request dans laquelle la version de cette dépendance a été mise à jour.

Lors de l'ouverture de Pull Request, le workflow 'validate-renovate.yml' est exécuté. Si le titre de la Pull Request ne commence pas par 'renovate/', l'exécution du workflow est bloquée.

2. Exécution des tests

Les workflows de tests ('playwright-tests.yml' et 'vitest-tests.yml') sont exécutés. 'validate-renovate.yml' récupère leur résultat.

3. Fermeture de la Pull Request

Si un ou plusieurs workflows de tests ont échoué, la Pull Request est automatiquement fermée.

4. Notification à l'équipe

Si tous les workflows de tests ont réussi, une notification est envoyée sur kChat à l'équipe grâce à une requête à l'API de notification :

(<https://notifications.backtik.ch/api/notifications/renovate>)

La notification contient le nom du dépôt Git, la dépendance à mettre à jour et sa version, et le lien de la Pull Request.

Génération des tests unitaires Vitest

Introduction

Cette fonctionnalité permet de générer automatiquement des suites de tests unitaires frontend Vitest pour les différents composants Vue.js et les composables.

Il s'agit d'une première version qui génère une première suite de tests. Ces tests doivent être refactorisés, corrigés et complétés.

Déclenchement

Les workflows de génération et d'exécution des tests peuvent être déclenchés manuellement ou par un autre workflow.

Le workflow d'exécution des tests est déclenché lors de l'exécution du workflow 'tests.yml', lui même déclenché lors des Pull Requests sur les branches 'main' ou 'preprod'.

Fichiers utilisés

- .github > workflows > generate-vitest.yml
- .github > workflows > vitest-tests.yml
- .github > workflows > tests.yml
- scripts > vitest-generator > analyze-project.mjs
- scripts > vitest-generator > generate-vitest.mjs
- scripts > vitest-generator > prompt.md

Génération des tests unitaires Vitest

Déroulement



1. Analyse des composants et composables

Lors de l'exécution du workflow de génération des tests, le script d'analyse extrait les fichiers de configuration importants, les composants et les composables, ainsi que leurs tests s'ils existent. Il en crée un fichier 'vite-test-context.json'.

2. Génération

Le script de génération construit la requête à partir du prompt et du fichier 'vite-test-context.json'. Après l'envoi de la requête, il récupère la réponse du LLM, la nettoie puis écrit les tests dans le dossier 'tests/Unit/vitest/'.

3. Exécution

Le workflow d'exécution des tests vitest exécute la commande 'npx vitest run'.

Génération des tests E2E Playwright

Introduction

Cette fonctionnalité permet de générer des suites de tests E2E Playwright grâce à l'outil Codegen, puis de les refactoriser automatiquement par IA.

Il s'agit d'une première version. Ces tests doivent être refactorisés, corrigés et complétés.

Déclenchement

Contrairement aux autres solutions, la génération des tests se fait semi-automatiquement. Seule la refactorisation des tests est automatique.

Le workflow d'exécution des tests est déclenché lors de l'exécution du workflow 'tests.yml', lui même déclenché lors des Pull Requests sur les branche 'main' ou 'preprod'.

Fichiers utilisés

- .github > workflows > playwright-tests.yml
- .github > workflows > tests.yml
- .claude/skills/playwright-cli/SKILL.md
- tests/e2e/ai/prompt.md
- tests/e2e/codegen/*.spec.js
- playwright.config.js

Génération des tests E2E Playwright

Déroulement



1. Génération des tests par Codegen

Codegen permet de générer les tests en interagissant avec l'interface. Il faut démarrer les serveurs de développement nécessaire, puis exécuter la commande suivante.

```
npx playwright codegen \  
  --output tests/e2e/codegen/<TITRE_TEST>.spec.js \  
  <APP_URL>
```

Après la fin de l'enregistrement du parcours utilisateur souhaités, le fichier de test est enregistré dans le dossier 'tests/e2e/codegen/'.

2. Refactorisation des tests par LLM

Pour démarrer la refactorisation des tests, il faut ouvrir son agent de codage (par ex. Claude Code) et lui transmettre l'instruction suivante :

```
Exécute les instructions de './tests/e2e/ai/prompt.md'
```

Le LLM analysera les fichiers de tests et les refactorisera pour supprimer les doublons, les erreurs humaines, les sélecteurs fragiles.

3. Exécution

Le workflow d'exécution des tests Playwright exécute la commande 'npx playwright test'.

Utilisation d'outils d'intelligence artificielle

Utilisation d'IA

Hormis la surveillance automatique des dépendances, toutes les solutions utilisent et dépendent d'un LLM.

Pour l'expérimentation, c'est le modèle gpt-5.2 d'OpenAI qui a principalement été utilisé grâce à son API.

Cependant, un autre modèle peut être utilisé après quelques modifications :

- Installation de la dépendance prévue par le fournisseur
- Remplacer 'BP_OPENAI_API_KEY' et 'BP_OPENAI_MODEL' dans les variables d'environnement et dans les secrets et variables du dépôt GitHub.
- Adapter les scripts (.mjs) au nouveau fournisseur et modèle.

Conseils d'utilisation

L'utilisation d'IA comporte certains risques tels qu'une mauvaise interprétation des consignes et du contexte, des erreurs, des modifications non souhaitées et d'obsolescence de ses données.

Pour cette raison, il est conseillé d'utiliser ce boilerplate en prenant quelques précautions :

- Enregistrer l'état du projet avant toute utilisation.
- Valider le contenu produit avant tout 'merge' ou 'push'.
- Vérifier qu'aucun fichier n'ait fait l'objet de modifications involontaires.

hacktik

